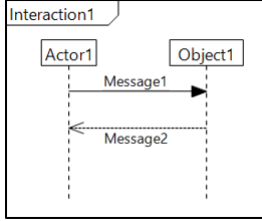
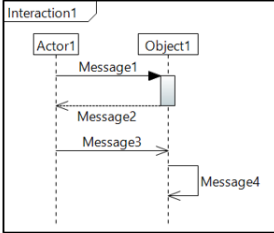
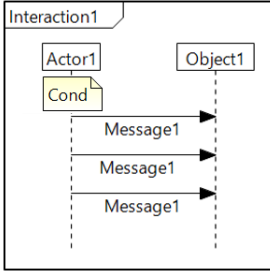


The rules are integrated into a module that handles both simple and structured interactions. The main rules are summarized in following tables (Tab. 1, Tab. 2)

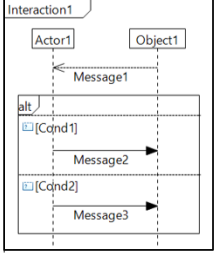
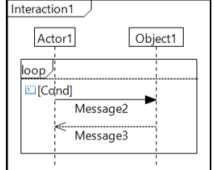
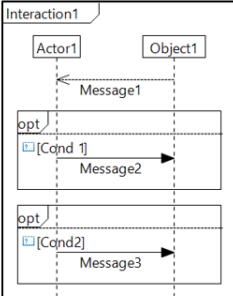
Table 1: Mapping UML_{1.5} Sequence Diagram to Maude Rewriting Rules

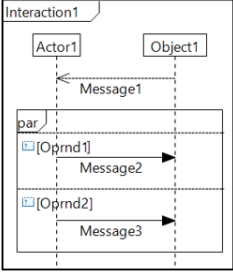
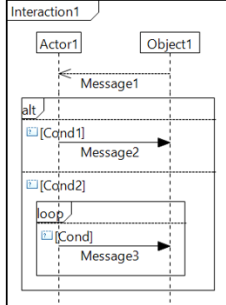
Interaction Structures	Corresponding Maude Rewriting Rules
 <pre> sequenceDiagram actor Actor1 participant Object1 Actor1->>Object1: Message1 Object1-->>Actor1: Message2 </pre>	<pre> r1 [UML15-Msg1] : [UML1.5,none,NoMsg] => [UML1.5,< (LL1:Object),{MM1:Synchron},(LL2:Actor)>, NoMsg;<(LL1:Object),{MM1:Synchron},(LL2:Actor)>] . r1 [UML15-Msg2] : [UML1.5,<(LL1:Object),{MM1:Synchron},(LL2:Actor)>,list1] => [UML1.5,<(LL2:Actor),{MM2:Reply},(LL1:Object)>, List1;<(LL2:Actor),{MM2:Reply},(LL1:Object)>] . </pre>
 <pre> sequenceDiagram actor Actor1 participant Object1 Actor1->>Object1: Message1 Object1-->>Actor1: Message2 Actor1->>Object1: Message3 Object1->>Object1: Message4 </pre>	<pre> r1 [UML15-Msg1] : [UML1.5,none,NoMsg] => [UML1.5,<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>, NoMsg;<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>] . r1 [UML15-Msg2] : [UML1.5,<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>,list2] => [UML1.5,<(Object1:Object),{Message2:Reply},(Actor1:Actor)>, list1;<(Object1:Object),{Message2:Reply},(Actor1:Actor)>] . r1 [UML15-Msg3] : [UML1.5,<(Object1:Object),{Message2:Reply},(Actor1:Actor)>,list2] => [UML1.5,<(Actor1:Actor),{Message3:Asynchron},(Object1:Object)>, list1;<(Actor1:Actor),{Message3:Asynchron},(Object1:Object)>] . r1 [UML15-Msg4] : [UML1.5,<(Actor1:Actor),{Message3:Asynchron},(Object1:Object)>,list2] => [UML1.5,<(Object1:Object),{Message4:SelfMessage},(Object1:Object)>, list1;<(Object1:Object),{Message4:SelfMessage},(Object1:Object)>] . </pre>
 <pre> sequenceDiagram actor Actor1 participant Object1 Actor1->>Object1: Message1 Note over Actor1: Cond Actor1->>Object1: Message1 Actor1->>Object1: Message1 </pre>	<pre> r1 [UML15-Msg1] : [UML1.5,none,NoMsg] => [UML1.5,<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>, NoMsg;<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>] . r1 [UML15-Msg2] : [UML1.5,<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>,list1] => [UML1.5,<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>, list1;<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>] . r1 [UML15-Msg3] : [UML1.5,<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>,list1] => [UML1.5,<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>, list1;<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>] . </pre>

	<pre> r1 [UML15-Msg1] : [UML1.5,none,NoMsg] =>[UML1.5,<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>, NoMsg;<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>]. r1 [UML15-Msg2] : [UML1.5,<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>,list1] =>[UML1.5,<(Actor1:Actor),{Message2:Synchron},(Object1:Object)>, list1;<(Actor1:Actor),{Message2:Synchron},(Object1:Object)>]. </pre>
	<pre> r1 [UML15-Msg1] : [UML1.5,none,NoMsg] =>[UML1.5,<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>, NoMsg;<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>]. r1 [UML15-Msg2] : [UML1.5,<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>,list1] =>[UML1.5,<(Actor1:Actor),{Message2:Synchron},(Object1:Object)>, list1;<(Actor1:Actor),{Message2:Synchron},(Object1:Object)>]. </pre>

Table 2: Mapping UML_{2.5} Sequence Diagram to Maude Rewriting Rules

Interaction Structures	Corresponding Maude Rewriting Rules
	<pre> r1 [UML25-Msg1] : [UML2.5, none, NoMsg] =>[UML2.5,<(LL1:Object),{MM1:Synchron},(LL2:Actor)>, NoMsg;<(LL1:Object),{MM1:Synchron},(LL2:Actor)>]. r1 [UML25-Msg2] : [UML2.5,<(LL1:Object),{MM1:Synchron},(LL2:Actor)>,list2] =>[UML2.5,<(LL2:Actor),{MM2:Reply},(LL1:Object)>, list2;<(LL2:Actor),{MM2:Reply},(LL1:Object)>]. </pre>
	<pre> r1 [UML25-Msg1] : [UML2.5,none,NoMsg] =>[UML2.5,<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>, NoMsg;<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>]. r1 [UML25-Msg2] : [UML2.5,<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>,list2] =>[UML2.5,<(Object1:Object),{Message2:Reply},(Actor1:Actor)>, list2;<(Object1:Object),{Message2:Reply},(Actor1:Actor)>]. r1 [UML25-Msg3] : [UML2.5,<(Object1:Object),{Message2:Reply},(Actor1:Actor)>,list2] =>[UML2.5,<(Actor1:Actor),{Message3:Asynchron},(Object1:Object)>, list2;<(Actor1:Actor),{Message3:Asynchron},(Object1:Object)>]. r1 [UML25-Msg4] : [UML2.5,<(Actor1:Actor),{Message3:Asynchron},(Object1:Object)>,list2] =>[UML2.5,<(Object1:Object),{Message4:SelfMessage},(Object1:Object)>, list2;<(Object1:Object),{Message4:SelfMessage},(Object1:Object)>]. </pre>

	<pre> r1[UML25-Msg1] : [UML2.5,none,NoMsg] =>[UML2.5,<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>, NoMsg;<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>]. r1[Alt-Fragment] : [UML2.5,<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>,list2] =>[UML2.5,Alt{Cond1:<(Actor1:Actor),{Message2:Asynchron},(Object1:Object) >}, list2;<(Actor1:Actor),{Message2:Asynchron},(Object1:Object)>]. r1[Alt-Fragment2] : [UML2.5,Alt{Cond1:<(Actor1:Actor),{Message2:Asynchron},(Object1:Object) >},list2] =>[UML2.5,Alt{Cond2:<(Actor1:Actor),{Message3:Asynchron},(Object1:Object) >}, list2;<(Actor1:Actor),{Message3:Asynchron},(Object1:Object)>]. </pre>
	<pre> r1[UML25-Msg1] : [UML2.5,none,NoMsg] r1[Loop-Fragment] : =>[UML2.5,Loop{1,2:Cond, [<(Actor1:Actor),{Message2:Asynchron},(Object1:Object)> (Object1:Object) ,{Message3:Reply},(Actor1:Actor)>]}, list2;<(Actor1:Actor),{Message2:Asynchron},(Object1:Object)>; <(Object1: Object),{Message3:Reply},(Actor1:Actor)>]. </pre>
	<pre> r1[UML25-Msg1] : [UML2.5,none,NoMsg] =>[UML2.5,<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>,NoMsg;<(Actor1:Actor),{Message1:Synchron},(Object1:Object)>]. r1[Opt-Fragment] : [UML2.5,<(Actor1:Actor),{Message1:Synchron},(Object1:Object) >,list2] =>[UML2.5,Opt{Cond1:<(Actor1:Actor),{Message2:Asynchron},(Object1:Object) >}, list2;<(Actor1:Actor),{Message2:Asynchron},(Object1:Object)>]. r1[Opt-Fragment1] : [UML2.5,Opt{Cond1:<(Actor1:Actor),{Message2:Asynchron},(Object1:Object) >},list2] =>[UML2.5,Opt{Cond2:<(Actor1:Actor),{Message3:Asynchron},(Object1:Object) >}, list2;<(Actor1:Actor),{Message3:Asynchron},(Object1:Object)>]. </pre>

	<pre> r1[UML25-Msg] : [UML2.5,none,NoMsg] =>[UML2.5,<(Actor1:Actor),{Message1:Synchron},{Object1:Object}>, NoMsg;<(Actor1:Actor),{Message1:Synchron},{Object1:Object}>]. r1[Par-Fragment] : [UML2.5,<(Actor1:Actor),{Message1:Synchron},{Object1:Object}>,list2] =>[UML2.5,Par{Opnd1:<(Actor1:Actor),{Message2:Asynchron},{Object1:Object}>}, list2;<(Actor1:Actor),{Message2:Asynchron},{Object1:Object}>]. r1[Par-Fragment1] : [UML2.5,Par{Opnd1:<(Actor1:Actor),{Message2:Asynchron},{Object1:Object}>, list2] =>[UML2.5,Par{Opnd2:<(Actor1:Actor),{Message3:Asynchron},{Object1:Object}>, list2;<(Actor1:Actor),{Message3:Asynchron},{Object1:Object}>]. </pre>
	<pre> r1 [UML25-Msg1] : [UML2.5,none,NoMsg] =>[UML2.5,<(Actor1:Actor),{Message1:Synchron},{Object1:Object}>, NoMsg;<(Actor1:Actor),{Message1:Synchron},{Object1:Object}>]. r1[Nested-Fragment] : [UML2.5,<(Actor1:Actor),{Message1:Synchron},{Object1:Object}>,list2] =>[UML2.5,Alt{Cond1:<(Actor1:Actor),{Message2:Asynchron},{Object1:Object}>, list2;<(Actor1:Actor),{Message2:Asynchron},{Object1:Object}>]. r1 [Nested-Loop] : [UML2.5,Alt{Cond1:<(Actor1:Actor),{Message2:Asynchron},{Object1:Object}>, list2] =>[UML2.5,Alt{Cond2:Loop{1,2:Cond,<(Actor1:Actor),{Message3:Asynchron},{ Object1:Object}>}}, list2;<(Actor1:Actor),{Message3:Asynchron},{Object1:Object}>]. </pre>